Mini Project Report on

TSDL-3-Java

Submitted to Vishwakarma University, Pune

By

Name : Snehal Late

Department of Computer Engineering

Faculty of Science and Technology

CONTENTS

STORE MANAGEMENT SYSTEM

INTRODUCTION

The Store Management System is a Java-based desktop application designed to facilitate the management of a store's inventory, customers, orders, and order items. This system provides a graphical user interface (GUI) built using Java Swing, allowing users to interact with the database seamlessly. The backend of the application is powered by a MySQL database, and JDBC is used for database connectivity.

It streamlines store operations by providing an intuitive interface for managing products, customers, orders, and order items. With its robust backend and user-friendly frontend, this application serves as a valuable tool for store administrators to maintain and track store-related data efficiently.

METHODOLOGY

Methods and Algorithms Used in the Store Management System:

1. Database Connection Establishment:

- Method: The `establishConnection()` method establishes a connection to the MySQL database using JDBC.

- Algorithm: JDBC's `DriverManager.getConnection()` method is utilized to connect to the database by providing the database URL, username, and password.

2. Adding Products:

- Method: The `addProduct()` method adds a new product to the database.

- Algorithm:

  - Retrieve product details (ID, name, price, quantity) entered by the user from text fields.

  - Construct an SQL INSERT query with placeholders for the product details.

  - Prepare a PreparedStatement object and set the values for the placeholders.

  - Execute the PreparedStatement to insert the product into the database.

  - Handle potential SQL and NumberFormatExceptions.

3. Adding Customers:

- Method: The `addCustomer()` method adds a new customer to the database.

- Algorithm:

- Retrieve customer details (ID, name, email) entered by the user from text fields.

- Construct an SQL INSERT query with placeholders for the customer details.

- Prepare a PreparedStatement object and set the values for the placeholders.

- Execute the PreparedStatement to insert the customer into the database.

- Handle potential SQL and NumberFormatExceptions.

4. Adding Orders:

- Method: The `addOrder()` method adds a new order to the database.

- Algorithm:

- Retrieve order details (ID, customer ID, total amount) entered by the user from text fields.

- Construct an SQL INSERT query with placeholders for the order details.

- Prepare a PreparedStatement object and set the values for the placeholders.

- Execute the PreparedStatement to insert the order into the database.

- Handle potential SQL and NumberFormatExceptions.

5. Adding Order Items:

- Method: The `addOrderItem()` method adds a new order item to the database.

- Algorithm:

- Retrieve order item details (ID, order ID, product ID, quantity, subtotal) entered by the user from text fields.

- Construct an SQL INSERT query with placeholders for the order item details.

- Prepare a PreparedStatement object and set the values for the placeholders.

- Execute the PreparedStatement to insert the order item into the database.

- Handle potential SQL and NumberFormatExceptions.

6. GUI Setup:

- Method: The `setupUI()` method configures the graphical user interface (GUI) components using Java Swing.

- Algorithm:

- Create panels for products, customers, orders, and order items.

- Add labels and text fields for inputting data.

- Create buttons for performing operations (e.g., adding products, customers, orders).

- Arrange components using GridBagLayout for a structured layout.

7. Main Method:

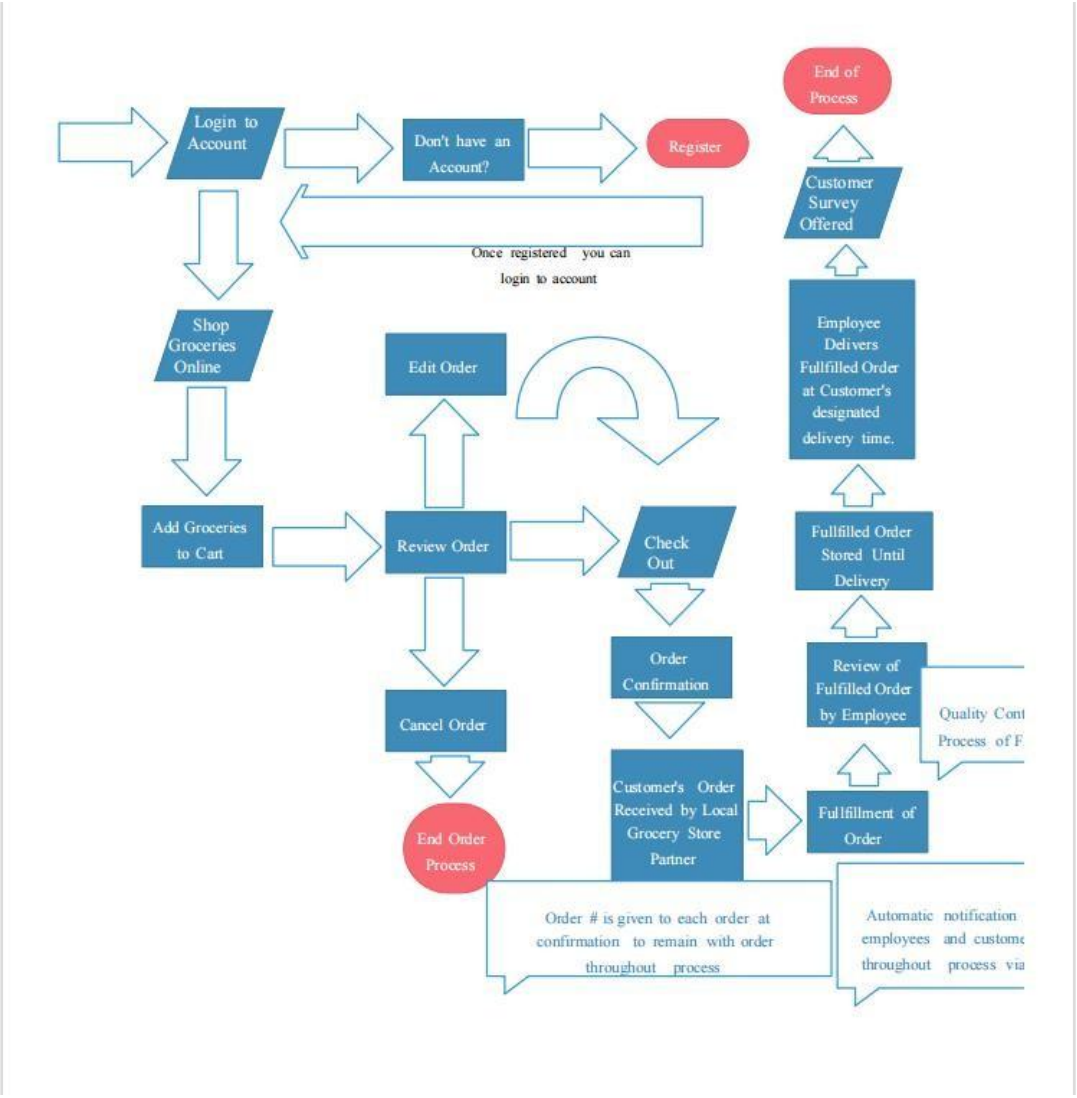
- Method: The `main()` method is the entry point of the application.

- Algorithm:

- Create an instance of the StoreManagementSystem class.

- Invoke the `invokeLater()` method of SwingUtilities to ensure GUI initialization on the event dispatch thread.

FLOWCHART



IMPLEMENTATION

Database Connection and GUI Initialization

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.sql.*;

public class StoreManagementSystem extends JFrame {
    // Database connection parameters
    private final String userName = "root";
    private final String password = "mysql_2024";
    private final String serverName = "localhost";
    private final int portNumber = 3306;
    private final String dbName = "store_db";

    // Table names
    private final String productsTableName = "products";
    private final String customersTableName = "customers";
    private final String ordersTableName = "orders";
    private final String orderItemsTableName = "order_items";

    // Database connection object
    private Connection conn;

    // GUI components
    private JTextField productIdField, productNameField, productPriceField,
productQuantityField;
    private JTextField customerIdField, customerNameField, customerEmailField;
    private JTextField orderIdField, orderCustomerIdField,
orderTotalAmountField;
    private JTextField orderItemIdField, orderItemOrderIdField,
orderItemProductIdField, orderItemQuantityField, orderItemSubtotalField;
    private JButton addProductButton, addCustomerButton, addOrderButton,
addOrderItemButton;
    private JTextArea outputArea;

    // Constructor
    public StoreManagementSystem() {
        // Establish database connection
        try {
            conn = DriverManager.getConnection("jdbc:mysql://" + serverName +
":" + portNumber + "/" + dbName,
                    userName, password);
        } catch (SQLException e)
            { e.printStackTrace();
            JOptionPane.showMessageDialog(this, "Failed to connect to the
database.");
            System.exit(1);
        }

        // Set up GUI components for products, customers, orders, and order
items
        // (omitted for brevity)
    }
```

This section initializes the Store Management System by establishing a connection to the MySQL database and setting up the graphical user interface (GUI) components.

Adding Products:

```
private void addProduct()
        { try {
            // Retrieve product details from text fields
            int id = Integer.parseInt(productIdField.getText());
            String name = productNameField.getText();
            double price = Double.parseDouble(productPriceField.getText());
            int quantity = Integer.parseInt(productQuantityField.getText());

            // Prepare SQL statement to insert product into the database
            String insertQuery = "INSERT INTO " + productsTableName + " (id,
name, price, quantity) VALUES (?, ?, ?, ?)";
            PreparedStatement statement = conn.prepareStatement(insertQuery);
            statement.setInt(1, id);
            statement.setString(2, name);
            statement.setDouble(3, price);
            statement.setInt(4, quantity);

            // Execute the SQL statement
            statement.executeUpdate();

            // Update the output area with success message
            outputArea.append("Product added: ID=" + id + ", Name=" + name + ",
Price=" + price + ", Quantity=" + quantity + "\n");
        } catch (SQLException | NumberFormatException ex) {
            // Display error message if there's an exception
            JOptionPane.showMessageDialog(this, "Error adding product: " +
ex.getMessage());
            ex.printStackTrace();
        }
    }
```

This method addProduct() is called when the user clicks the "Add Product" button. It retrieves the product details entered by the user from the text fields, prepares an SQL INSERT statement, executes the statement to add the product to the database, and updates the output area with a success message. If there's an error, it displays an error message.

Methods (addCustomer(), addOrder(), addOrderItem()) follow a similar structure to addProduct(). They retrieve the respective details from the text fields, prepare SQL INSERT statements, execute the statements, and update the output area accordingly.

Main Method:

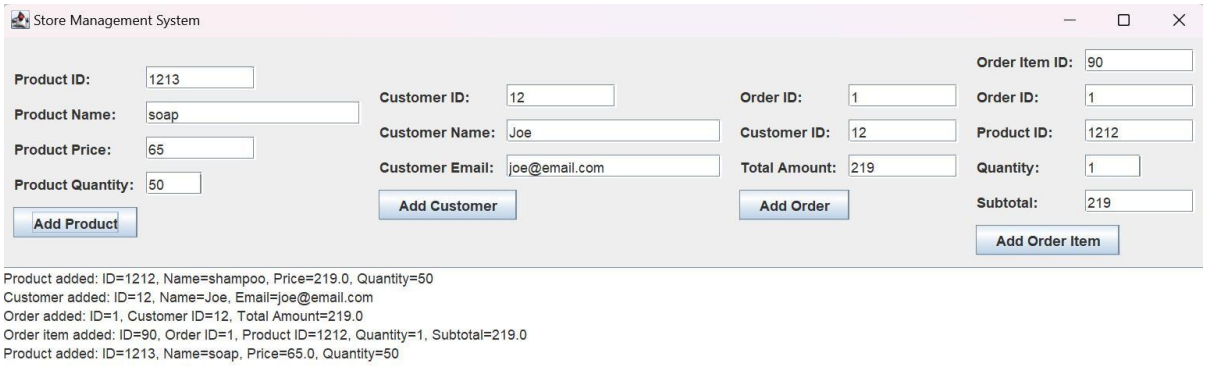
```
  // Main method to start the application
    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
```

```
            public void run() {
                new StoreManagementSystem();
            }
        });
    }
}
```

This is the entry point of the application. It creates an instance of the StoreManagementSystem class and displays the GUI on the Event Dispatch Thread (EDT) using SwingUtilities.invokeLater().

This breakdown provides a detailed explanation of the main functionalities and structure of the Store Management System code. Each section describes the purpose and functionality of the corresponding code snippet.

RESULTS




```
mysql> select * from products;
+--------+------------+----------+-------------+
| id     | name       | price    | quantity    |
+--------+------------+----------+-------------+
|   20   | mirinda    |  75.00   |       2     |
|   23   | apple      | 100.00   |      50     |
| 1200   | veggies    | 120.00   |      10     |
```

```
| 1212 | shampoo | 219.00 |      50 |
| 1213 | soap    |  65.00 |      50 |
| 2001 | lays    |  20.00 |       2 |
| 2002 | kurkure |  20.00 |       5 |
| 2003 | chips   |  20.00 |      10 |
| 2007 | Balaji  |  20.00 |       5 |
+--------+------------+----------+-------------+
9 rows in set (0.00 sec)

mysql> select * from order_items;
+-----+-------------+---------------+-------------+-------------+
| id | order_id | product_id | quantity | subtotal |
+-----+-------------+---------------+-------------+-------------+
| 90 |       1 |      1212 |       1 |   219.00 |
+-----+-------------+---------------+-------------+-------------+
1 row in set (0.00 sec)

mysql> select * from orders;
+-----+-----------------+------------------+---------------------------+
| id | customer_id | total_amount | order_date          |
+-----+-----------------+------------------+---------------------------+
|  1 |        12 |      219.00 | 2024-04-18 21:02:57 |
+-----+-----------------+------------------+---------------------------+
1 row in set (0.00 sec)

mysql> select * from customers;
+-----+--------+--------------------+
| id | name | email          |
+-----+--------+--------------------+
| 12 | Joe  | joe@email.com |
+-----+--------+--------------------+
```

```
1 row in set (0.00 sec)

mysql> select * from products;
+--------+------------+----------+-------------+
| id   | name     | price  | quantity |
+--------+------------+----------+-------------+
|  20 | mirinda |  75.00 |       2 |
|  23 | apple    | 100.00 |      50 |
| 1200 | veggies | 120.00 |      10 |
| 1212 | shampoo | 219.00 |      50 |
| 1213 | soap     |  65.00 |      50 |
| 2001 | lays     |  20.00 |       2 |
| 2002 | kurkure |  20.00 |       5 |
| 2003 | chips    |  20.00 |      10 |
| 2007 | Balaji  |  20.00 |       5 |
+--------+------------+----------+-------------+
9 rows in set (0.00 sec)
```

CONCLUSION

The Store Management System project has achieved several key findings:

Database Integration: Successfully integrated a MySQL database with the Java application to store and manage data related to products, customers, orders, and order items.

Graphical User Interface (GUI): Developed a user-friendly GUI using Swing components in Java, allowing users to easily interact with the system and perform various operations such as adding products, customers, orders, and order items.

CRUD Operations: Implemented functionality for performing CRUD (Create, Read, Update, Delete) operations on the database tables, enabling users to add, view, update, and delete records as needed.

Exception Handling: Implemented robust error handling mechanisms to gracefully handle exceptions such as database connection failures, invalid input data, or SQL errors, ensuring smooth operation of the application.

Learning Experience: Throughout the project, valuable lessons were learned regarding database connectivity, GUI development, SQL queries, and error handling techniques in Java. Understanding these concepts has enhanced my knowledge and skills in software development.

The significance of this work lies in its practical application in real-world scenarios, particularly in retail businesses or stores. By providing a streamlined system for managing inventory, customers, and orders, the Store Management System can help businesses improve efficiency, reduce errors, and enhance customer satisfaction..